Decentralization is one of the cornerstones of the data mesh. The most effective and scalable way to achieve decentralization is to base it on domains. The concept of the domain in a data mesh is based on DDD. When you better understand DDD, it will be easier to apply these concepts in the data field.

DDD is a software design approach that emphasizes common models and language between business and development teams.

At Messflix, the business people call a paying customer a *subscriber*. Therefore, following DDD principles, the Yellow team developing Messflix Streaming Platform uses the word *subscriber* in its internal conversations, requirements, documentation, and code (e.g., class names).

Achieving a shared understanding of a business, its rules, and policies is at the heart of the DDD approach. Eric Evans's *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Addison-Wesley) introduced that approach in 2003. DDD revolutionized software development. It promoted the idea that we need to understand the problem space to create and model a solution effectively. So it makes sense, right?

DDD as a methodology can be divided into two types of patterns:

- *Tactical patterns*—Design patterns for object-oriented programming
- *Strategic patterns*—Patterns focused on domains, dividing responsibility and ownership, and communication (between systems and people responsible for those systems)

From the perspective of the data mesh, strategic patterns are more relevant, because the goal is to design ownership, responsibilities, and communication. Let's learn how we can utilize DDD domain concepts in practice.

4.1.2 Invite the right people

You now know what the domain is. You also know that one of DDD's goals is to set up a common language. But who should you find to discover that